

Indexing in DBMS

These notes are written for placement preparation. They cover the major index types, how they work internally, when to use them, their advantages and disadvantages, and common interview points.

1. What Is Indexing?

An index is an auxiliary data structure that helps the DBMS find rows faster without scanning the entire table.

Without an index, the DBMS may perform a full table scan:

```
SELECT * FROM Student WHERE roll_no = 105;
```

If the table has 1,000,000 rows and no useful index exists, the DBMS may have to check many or all rows.

With an index on `roll_no`, the DBMS can directly locate the matching row or a small set of matching rows.

An index is similar to the index at the back of a textbook:

- The book contains the actual content.
- The index stores keywords in sorted order.
- Each keyword points to the page where the content exists.

In a DBMS:

- The table contains the actual records.
- The index stores search key values.
- Each search key value points to one or more table records.

2. Why Indexes Are Needed

Indexes improve the performance of:

- Search queries using `WHERE`
- Range queries using `<`, `>`, `BETWEEN`
- Sorting using `ORDER BY`
- Grouping using `GROUP BY`
- Joins using foreign keys or commonly joined columns
- Uniqueness checks using `PRIMARY KEY` or `UNIQUE`

Example:

```
SELECT * FROM Employee WHERE department_id = 10;
SELECT * FROM Employee WHERE salary BETWEEN 50000 AND 90000;
SELECT * FROM Employee ORDER BY joining_date;
```

All these queries can become faster if suitable indexes exist.

3. Basic Terms

Data File

The actual file or storage area where table records are stored.

Index File

A separate structure that stores search key values and pointers to actual records or data blocks.

Search Key

The column or group of columns on which the index is built.

Example:

```
CREATE INDEX idx_student_name ON Student(name);
```

Here, **name** is the search key.

Pointer

A reference to the actual record or disk block. It may point to:

- A row
- A page/block
- A bucket
- Another index node

Data Block / Page

Databases store rows in fixed-size pages or blocks, not one row at a time. A disk I/O usually reads or writes one page.

Indexing mainly reduces the number of pages that must be read.

4. Cost Without Indexing

Suppose a table has **N** records.

For a sequential search:

- Average search cost: about **N / 2** records
- Worst-case search cost: **N** records

If records are stored across many disk blocks, this can mean many expensive disk reads.

For large tables, full scans are costly.

5. Cost With Indexing

With a tree-based index such as a B+ tree:

- Search cost is usually $O(\log n)$
- The tree height is small because each node stores many keys
- In real databases, the index height is often 2 to 4 levels for very large tables

With a hash index:

- Equality search can be close to $O(1)$ on average
- Range search is poor because hash values are not stored in sorted order

6. Advantages of Indexing

- Faster search
- Faster joins
- Faster sorting if index order matches query order
- Faster grouping in some cases
- Faster duplicate checks for unique constraints
- Efficient range queries with ordered indexes
- Lower disk I/O for selective queries

7. Disadvantages of Indexing

Indexes are not free.

- They require extra storage space.
- Insert operations become slower because indexes must be updated.
- Delete operations become slower because index entries must be removed.
- Update operations become slower if indexed columns are changed.
- Too many indexes can confuse the optimizer or increase maintenance cost.
- Indexes on low-selectivity columns may not help much.

Example of a low-selectivity column:

```
gender: M/F  
is_active: true/false
```

If half the table matches a condition, an index may not be useful because the DBMS still has to read many rows.

8. When an Index Is Useful

An index is usually useful when:

- The table is large.
- The query retrieves a small percentage of rows.
- The column is frequently used in **WHERE**.

- The column is frequently used in joins.
- The column is frequently used in `ORDER BY`.
- The column has high selectivity.
- Range queries are common.

High selectivity means the column has many distinct values.

Good examples:

- `student_id`
- `email`
- `roll_no`
- `account_number`
- `order_id`

Less useful examples:

- `gender`
- `is_deleted`
- `status` with only a few possible values

9. When an Index May Not Be Useful

An index may not help when:

- The table is very small.
- The query returns a large portion of the table.
- The indexed column has many duplicate values.
- The query applies functions to the indexed column.
- The query pattern does not match the index order.

Example where normal index may not be used:

```
SELECT * FROM Student WHERE LOWER(name) = 'rahul';
```

If the index is on `name`, but the query uses `LOWER(name)`, the DBMS may not use the index unless a function-based index exists.

10. Primary Index

A primary index is an index built on the primary key or on the ordering key of a file.

In many DBMS explanations, a primary index is built on a field that:

- Is unique
- Determines the physical or logical order of records

Example:

```
CREATE TABLE Student (  
    roll_no INT PRIMARY KEY,  
    name VARCHAR(50),  
    branch VARCHAR(20)  
);
```

The DBMS usually creates an index automatically on the primary key `roll_no`.

Properties of Primary Index

- Usually built on a unique key.
- Usually has one index entry per block in sparse indexing.
- Records are often stored sorted by the primary key in theoretical file organization.
- Helps fast access using the primary key.
- Ensures uniqueness if associated with a primary key constraint.

Example

Data file sorted by `roll_no`:

```
Block 1: 101, 102, 103  
Block 2: 104, 105, 106  
Block 3: 107, 108, 109
```

Primary index:

```
101 -> Block 1  
104 -> Block 2  
107 -> Block 3
```

If searching for `105`, the DBMS:

1. Searches the index.
2. Finds the greatest key less than or equal to `105`, which is `104`.
3. Goes to Block 2.
4. Searches inside Block 2.

Important Point

Primary indexes are commonly sparse because the data file is sorted on the primary key. One index entry per block is enough.

11. Secondary Index

A secondary index is an index built on a column that is not the primary ordering field of the data file.

Example:

```
CREATE INDEX idx_student_branch ON Student(branch);
```

Here, **branch** is not necessarily the primary key. The index helps queries like:

```
SELECT * FROM Student WHERE branch = 'CSE';
```

Properties of Secondary Index

- Can be built on non-key attributes.
- Can be built on unique or non-unique columns.
- Usually dense because records are not physically ordered by the secondary key.
- There can be many secondary indexes on a table.
- Useful for alternate access paths.

Example

Student table:

roll_no	name	branch
101	Aman	CSE
102	Neha	ECE
103	Ravi	CSE
104	Meena	ME

Secondary index on **branch**:

```
CSE -> 101, 103  
ECE -> 102  
ME -> 104
```

Since many students may have the same branch, the index may point to a list of record pointers.

Secondary Index on a Candidate Key

If the secondary index is built on a unique column such as **email**, each key points to one record.

```
email -> record pointer
```

Secondary Index on Non-Key Column

If the secondary index is built on a non-unique column such as `department_id`, each key may point to many records.

```
department_id -> list of record pointers
```

12. Primary Index vs Secondary Index

Basis	Primary Index	Secondary Index
Built on	Primary key or ordering key	Non-ordering attribute
Uniqueness	Usually unique	May be unique or non-unique
Physical order	Data file may be sorted on this key	Data file usually not sorted on this key
Number allowed	Usually one primary ordering index	Many secondary indexes possible
Dense or sparse	Often sparse	Usually dense
Storage	Less storage if sparse	More storage if dense
Example	Index on <code>roll_no</code>	Index on <code>branch</code>

13. Dense Index

A dense index has an index entry for every search key value in the data file.

If the search key is unique, there is one index entry for every record.

```
101 -> pointer to record 101
102 -> pointer to record 102
103 -> pointer to record 103
104 -> pointer to record 104
```

If the search key is not unique, there may be one index entry for every distinct search key value, and each entry points to a list of records.

```
CSE -> pointers to all CSE records
ECE -> pointers to all ECE records
ME  -> pointers to all ME records
```

Advantages of Dense Index

- Faster direct lookup.
- Works even when data file is not sorted.
- Useful for secondary indexes.
- Can handle non-unique search keys.

Disadvantages of Dense Index

- Requires more storage.
- More expensive to maintain during insert/delete/update.
- Index file can become large.

When Dense Index Is Used

- Secondary indexes
- Indexes on non-ordering fields
- Unique indexes when direct row pointers are needed
- Cases where data file is not sorted on the search key

14. Sparse Index

A sparse index has index entries only for some search key values.

Usually, it has one index entry per data block.

Example:

```
Block 1: 101, 102, 103
Block 2: 104, 105, 106
Block 3: 107, 108, 109
```

Sparse index:

```
101 -> Block 1
104 -> Block 2
107 -> Block 3
```

To search for 105:

1. Find the largest index key less than or equal to 105.
2. The key is 104.
3. Go to the block pointed to by 104.
4. Search inside that block.

Important Condition

A sparse index requires the data file to be sorted on the search key.

If the file is not sorted, a sparse index cannot safely locate missing key values.

Advantages of Sparse Index

- Smaller index size.
- Less storage.

- Less maintenance overhead.
- Efficient when data is sorted.

Disadvantages of Sparse Index

- Slightly slower than dense index for exact lookup because the block must be searched.
- Cannot be used effectively if data is not sorted on the search key.
- Usually supports only one ordering path.

15. Dense Index vs Sparse Index

Basis	Dense Index	Sparse Index
Entries	Entry for every record or every search key value	Entry for some keys, usually one per block
Storage	More	Less
Lookup	Faster direct lookup	Requires block search after index lookup
Data file order	Not required	Required
Common use	Secondary indexes	Primary indexes
Maintenance	Higher	Lower
Example	Every <code>roll_no</code> has entry	First <code>roll_no</code> of each block has entry

16. Clustered Index

A clustered index determines the physical order of data rows in the table.

If a table has a clustered index on `roll_no`, rows are physically stored in the order of `roll_no`, or at least the storage is organized close to that order.

Example:

```
CREATE CLUSTERED INDEX idx_student_roll ON Student(roll_no);
```

Not all DBMSs use this exact syntax. SQL Server uses clustered indexes explicitly. MySQL InnoDB clusters data by the primary key.

Properties of Clustered Index

- The table data is stored according to the clustered index key.
- There can usually be only one clustered index per table.
- Very good for range queries.
- Very good for sorting by the clustered key.
- Insertions may be costly if new rows must be placed in the middle.
- The leaf level of the clustered index may contain the actual data rows.

Example

If data is clustered by `roll_no`:

```
101 Aman
102 Neha
103 Ravi
104 Meena
```

Query:

```
SELECT * FROM Student
WHERE roll_no BETWEEN 101 AND 104;
```

This is efficient because matching rows are stored close together.

Advantages of Clustered Index

- Excellent for range queries.
- Reduces random I/O because related rows are stored together.
- Useful for columns often used in ordering or range filtering.
- Can make queries faster when many consecutive rows are needed.

Disadvantages of Clustered Index

- Only one clustered index is possible because rows can have only one physical order.
- Insert/update can be slower if the clustered key changes or causes page splits.
- Bad clustered key choice can hurt performance.
- Wide clustered keys can make secondary indexes larger in some DBMSs.

Good Clustered Index Candidates

- Primary key
- Date/time column for time-series queries
- Frequently used range column
- Stable column that rarely changes
- Narrow column
- Increasing column such as auto-increment ID, depending on workload

Bad Clustered Index Candidates

- Frequently updated column
- Very wide column
- Random values such as random UUIDs, because they can cause page splits
- Low-selectivity column if range access is not useful

17. Non-Clustered Index

A non-clustered index is separate from the actual table data. It stores the index key and a pointer to the actual row.

The physical order of table rows does not follow the non-clustered index order.

Example:

```
CREATE INDEX idx_student_name ON Student(name);
```

Properties of Non-Clustered Index

- Does not determine physical order of data rows.
- A table can have many non-clustered indexes.
- Leaf nodes contain row pointers or primary key values, depending on DBMS.
- Useful for alternate search paths.
- May require an extra lookup to fetch full row data.

Example

Table stored by `roll_no`:

```
101 Aman
102 Neha
103 Ravi
104 Meena
```

Non-clustered index on `name`:

```
Aman -> row 101
Meena -> row 104
Neha -> row 102
Ravi -> row 103
```

The index is sorted by `name`, but the table is not.

Advantages of Non-Clustered Index

- Many can exist on one table.
- Useful for different query patterns.
- Good for equality lookups.
- Can support covering indexes.

Disadvantages of Non-Clustered Index

- Extra storage required.
- Extra row lookup may be needed.

- Slower than clustered index for large range scans if many random row lookups are required.
- Maintenance overhead during writes.

18. Clustered vs Non-Clustered Index

Basis	Clustered Index	Non-Clustered Index
Data order	Determines physical/logical row order	Separate from data order
Number per table	Usually one	Many
Leaf level	Often contains actual data rows	Contains pointers or row identifiers
Range queries	Very efficient	Can be less efficient if many row lookups needed
Storage	Table itself organized as index	Separate structure
Insert cost	Can be higher due to page splits	Also has maintenance cost
Example	Primary key in InnoDB	Index on <code>name</code> , <code>email</code> , <code>department_id</code>

19. Ordered Index

An ordered index stores search keys in sorted order.

Examples:

- B-tree index
- B+ tree index
- Dense ordered index
- Sparse ordered index

Ordered indexes are useful for:

- Equality search
- Range search
- Sorting
- Minimum/maximum queries
- Prefix matching in composite indexes

Example:

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'
ORDER BY order_date;
```

A B+ tree index on `order_date` can help because keys are ordered.

20. B-Tree

A B-tree is a balanced multi-way search tree used for indexing.

It keeps keys sorted and ensures that all leaf nodes are at the same level. This means search, insert, and delete operations take logarithmic time.

The "B" in B-tree is commonly explained as "Balanced", though the exact historical meaning is debated.

Why B-Trees Are Used in DBMS

Binary search trees are not ideal for disk storage because each node has only up to two children. This can make the tree tall, causing many disk I/O operations.

B-trees are better because each node can contain many keys and many child pointers.

This makes the tree short and disk-friendly.

Structure of a B-Tree Node

A B-tree node contains:

- Multiple keys
- Multiple child pointers
- Sometimes record pointers

Example node:

```
[10 | 20 | 30]
```

This node can divide values into ranges:

```
less than 10  
10 to 20  
20 to 30  
greater than 30
```

B-Tree Properties

For a B-tree of order m :

- Each node can have at most m children.
- Each node can have at most $m - 1$ keys.
- Except root, each internal node has at least $\lceil m / 2 \rceil$ children.
- Except root, each node has at least $\lceil m / 2 \rceil - 1$ keys.
- Root has at least one key if the tree is non-empty.
- All leaves appear at the same level.
- Keys inside each node are sorted.

B-Tree Search

To search for key **K**:

1. Start at root.
2. Search within the root node.
3. If **K** is found, return result.
4. If not found, follow the correct child pointer.
5. Repeat until key is found or leaf is reached.

B-Tree Insertion

1. Find the correct leaf position.
2. Insert the key in sorted order.
3. If the node overflows, split it.
4. Promote the middle key to the parent.
5. If the parent overflows, split recursively.
6. If the root overflows, create a new root.

B-Tree Deletion

Deletion is more complex:

1. Find the key.
2. Delete it.
3. If node underflows, borrow from sibling if possible.
4. If borrowing is not possible, merge with sibling.
5. Adjust parent keys.
6. If root becomes empty, reduce tree height.

Advantages of B-Tree

- Balanced height.
- Good for disk-based storage.
- Efficient equality search.
- Efficient range search compared with hashing.
- Handles dynamic insertions and deletions.
- Avoids long chains of nodes.

Disadvantages of B-Tree

- More complex than binary search tree.
- Range scanning is not as efficient as B+ tree because all leaves may not be linked.
- Internal nodes may store actual record pointers, reducing fanout.

21. B+ Tree

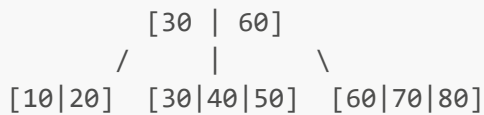
A B+ tree is a variation of the B-tree and is the most commonly used index structure in DBMSs.

In a B+ tree:

- Internal nodes store only search keys and child pointers.
- Actual data pointers are stored only at the leaf level.
- Leaf nodes are linked together in sorted order.

This makes B+ trees excellent for range queries.

B+ Tree Structure



The leaf nodes are linked:

```
[10|20] <-> [30|40|50] <-> [60|70|80]
```

Properties of B+ Tree

- All actual records or record pointers are at leaf nodes.
- Internal nodes only guide the search.
- Leaf nodes are linked.
- All leaves are at the same level.
- Keys are sorted.
- The tree remains balanced.
- High fanout because internal nodes store only keys and child pointers.

B+ Tree Search

1. Start at root.
2. Use internal keys to choose the correct child.
3. Continue until a leaf node is reached.
4. Search the leaf for the key.

Unlike B-tree, even if a key appears in an internal node, the search usually continues to the leaf because actual data pointers are stored at leaves.

B+ Tree Range Query

For:

```
SELECT * FROM Employee
WHERE salary BETWEEN 50000 AND 90000;
```

The DBMS:

1. Finds the first leaf containing 50000.
2. Scans linked leaf nodes sequentially.
3. Stops after passing 90000.

This is very efficient because leaf nodes are sorted and linked.

B+ Tree Insertion

1. Insert key in the correct leaf.
2. If the leaf overflows, split the leaf.
3. Copy the first key of the new leaf to the parent.
4. If parent overflows, split internal node.
5. Continue upward if required.

In B+ trees, the promoted key is copied to the parent, not removed from the leaf.

B+ Tree Deletion

1. Delete key from the leaf.
2. If the leaf still has enough keys, stop.
3. If leaf underflows, borrow from sibling if possible.
4. If borrowing is not possible, merge with sibling.
5. Update parent separator keys.
6. Continue upward if needed.

Advantages of B+ Tree

- Excellent for range queries.
- Excellent for ordered traversal.
- High fanout, so tree height is small.
- Stable logarithmic search time.
- Leaf-level linked list supports efficient sequential scan.
- Internal nodes are smaller because they do not store data pointers.
- Widely used in real DBMS indexing.

Disadvantages of B+ Tree

- More complex implementation.
- Extra storage for linked leaf pointers.
- Equality search may require going all the way to the leaf.
- Insert/delete operations may cause splits and merges.

22. B-Tree vs B+ Tree

Basis	B-Tree	B+ Tree
Data pointers	Can be in internal or leaf nodes	Only leaf nodes store data pointers
Search	May stop at internal node	Usually goes to leaf
Leaf linking	Not necessarily linked	Leaves are linked

Basis	B-Tree	B+ Tree
Range queries	Good	Excellent
Fanout	Lower if data pointers are in internal nodes	Higher
Tree height	May be slightly higher	Usually smaller
Sequential access	Less efficient	Very efficient
DBMS usage	Used, but less common	Very common

Why DBMSs Prefer B+ Trees

DBMSs usually prefer B+ trees because:

- Disk I/O is reduced due to high fanout.
- Range scans are extremely efficient.
- Sequential access is easy because leaves are linked.
- Internal nodes are compact.
- The structure works well with page-based storage.

23. Hash Index

A hash index uses a hash function to map search key values to buckets.

Example:

```
hash(key) = bucket number
```

If we search for `roll_no = 105`, the DBMS computes:

```
hash(105) = 3
```

Then it directly checks bucket 3.

Structure

```
Key -> Hash function -> Bucket -> Records
```

Example:

```
hash(101) = 1
hash(102) = 2
```

```
hash(103) = 3  
hash(104) = 1
```

Buckets:

```
Bucket 1: 101, 104  
Bucket 2: 102  
Bucket 3: 103
```

Hash Function

A hash function should:

- Distribute keys uniformly.
- Be fast to compute.
- Minimize collisions.
- Produce valid bucket addresses.

Collision

A collision occurs when two different keys map to the same bucket.

Example:

```
hash(101) = 1  
hash(104) = 1
```

Both keys go to Bucket 1.

Collision Handling

Common methods:

- Chaining: bucket points to a linked list or overflow pages.
- Open addressing: find another empty slot using probing.
- Overflow buckets: extra buckets store collided records.

In DBMS storage, overflow pages are common.

Advantages of Hash Index

- Very fast for equality search.
- Average lookup can be close to $O(1)$.
- Good for exact-match queries.
- Simple conceptually.

Example:

```
SELECT * FROM Employee WHERE employee_id = 501;
```

Disadvantages of Hash Index

- Poor for range queries.
- Cannot efficiently support sorting.
- Cannot efficiently support prefix search.
- Performance degrades with many collisions.
- May require rehashing or overflow management.

Bad query for hash index:

```
SELECT * FROM Employee  
WHERE salary BETWEEN 50000 AND 90000;
```

Hashing does not preserve order, so the DBMS cannot easily scan salaries in sorted order.

Hash Index Use Cases

Good for:

- Equality search
- Primary key lookup
- Unique lookup
- Exact match on high-selectivity columns

Not good for:

- Range queries
- ORDER BY
- GROUP BY requiring sorted order
- Prefix search such as LIKE 'abc%'
- Minimum or maximum search

24. Static Hashing

In static hashing, the number of buckets is fixed.

Example:

```
bucket = key % 10
```

Problem

If the table grows, buckets may overflow heavily.

If the table shrinks, many buckets may remain empty.

Advantages

- Simple.
- Fast when data size is stable.

Disadvantages

- Poor adaptability.
- Overflow chains can become long.
- Reorganization may be expensive.

25. Dynamic Hashing

Dynamic hashing allows the number of buckets to grow or shrink.

Common techniques:

- Extendible hashing
- Linear hashing

Extendible Hashing

Extendible hashing uses a directory of bucket pointers.

Important terms:

- Global depth: number of hash bits used by the directory.
- Local depth: number of hash bits used by a bucket.
- Bucket split: occurs when a bucket overflows.
- Directory doubling: occurs when local depth exceeds global depth.

Linear Hashing

Linear hashing grows gradually without a full directory doubling every time.

It splits buckets in a controlled sequence.

26. B+ Tree Index vs Hash Index

Basis	B+ Tree Index	Hash Index
Key order	Maintains sorted order	Does not maintain order
Equality search	Fast	Very fast on average
Range search	Excellent	Poor
Sorting	Can help	Cannot help
Prefix search	Can help	Usually cannot help
Min/max	Efficient	Not efficient

Basis	B+ Tree Index	Hash Index
Collision issue	No hash collision issue	Collisions must be handled
Common DBMS use	General-purpose default	Special-purpose exact lookup

Interview Answer

If a query needs equality lookup only, a hash index can be faster. If the query needs range search, sorting, or ordered traversal, a B+ tree is better.

27. Composite Index

A composite index is an index built on multiple columns.

Example:

```
CREATE INDEX idx_student_branch_sem
ON Student(branch, semester);
```

The search key is:

```
(branch, semester)
```

Why Composite Indexes Are Used

Composite indexes are useful when queries filter or sort using multiple columns together.

Example:

```
SELECT * FROM Student
WHERE branch = 'CSE' AND semester = 5;
```

An index on (branch, semester) can help.

28. Order Matters in Composite Indexes

The order of columns in a composite index is very important.

Index:

```
CREATE INDEX idx_emp_dept_salary
ON Employee(department_id, salary);
```

This index is sorted first by department_id, then by salary within each department.

It can help:

```
WHERE department_id = 10
```

It can help:

```
WHERE department_id = 10 AND salary > 50000
```

It usually cannot fully help:

```
WHERE salary > 50000
```

because `department_id` is the leftmost column and is missing.

29. Leftmost Prefix Rule

For a composite B+ tree index, the DBMS can efficiently use the leftmost prefix of the index.

Index:

```
(A, B, C)
```

Can be used for:

```
A  
A, B  
A, B, C
```

Usually cannot be fully used for:

```
B  
C  
B, C  
A, C
```

For `A`, `C`, the DBMS can use the index for `A`, but not efficiently for `C` unless other optimizer techniques apply.

Example

Index:

```
CREATE INDEX idx_orders_customer_date_status  
ON Orders(customer_id, order_date, status);
```

Useful:

```
WHERE customer_id = 101  
  
WHERE customer_id = 101  
  AND order_date >= '2026-01-01'  
  
WHERE customer_id = 101  
  AND order_date >= '2026-01-01'  
  AND status = 'SHIPPED'
```

Less useful:

```
WHERE order_date >= '2026-01-01'  
  
WHERE status = 'SHIPPED'
```

30. Composite Index and Range Conditions

In a composite index, once a range condition is used, columns after the range condition may be less useful for search.

Index:

```
(department_id, salary, age)
```

Query:

```
WHERE department_id = 10  
  AND salary > 50000  
  AND age = 25
```

The index can efficiently use:

```
department_id = 10  
salary > 50000
```

But `age = 25` may not be used as efficiently for direct navigation because salary is a range condition.

This depends on the DBMS optimizer, but it is a good interview rule.

31. Covering Index

A covering index is an index that contains all columns required by a query.

Example:

```
CREATE INDEX idx_emp_dept_name
ON Employee(department_id, name);
```

Query:

```
SELECT name
FROM Employee
WHERE department_id = 10;
```

The DBMS can answer this query using only the index, without reading the actual table.

This is called an index-only scan.

Advantages

- Avoids table lookup.
- Reduces disk I/O.
- Very fast for read-heavy queries.

Disadvantage

- More storage.
- More maintenance cost.
- Index may become wide if too many columns are included.

32. Unique Index

A unique index ensures that no duplicate values exist in the indexed column or columns.

Example:

```
CREATE UNIQUE INDEX idx_user_email
ON Users(email);
```

This prevents two users from having the same email.

Primary keys and unique constraints are usually implemented using unique indexes.

33. Multilevel Index

If an index file itself becomes large, the DBMS can build another index on top of the index.

This is called a multilevel index.

Example:

```
Level 2 index -> points to Level 1 index blocks
Level 1 index -> points to data blocks
Data file
```

B-trees and B+ trees are natural multilevel index structures.

Why Multilevel Indexing Is Useful

- Reduces search cost.
- Keeps top levels small.
- Root may remain in memory.
- Reduces disk I/O.

34. Indexing and SQL Examples

Create an index:

```
CREATE INDEX idx_employee_department
ON Employee(department_id);
```

Create a unique index:

```
CREATE UNIQUE INDEX idx_employee_email
ON Employee(email);
```

Create a composite index:

```
CREATE INDEX idx_employee_dept_salary
ON Employee(department_id, salary);
```

Drop an index:

```
DROP INDEX idx_employee_department;
```

Syntax can differ between DBMSs. For example, MySQL sometimes uses:

```
ALTER TABLE Employee DROP INDEX idx_employee_department;
```

35. Query Patterns and Best Index Choices

Query Pattern	Good Index
WHERE id = ?	Hash or B+ tree on id
WHERE salary BETWEEN ? AND ?	B+ tree on salary
ORDER BY created_at	B+ tree on created_at
WHERE email = ?	Unique B+ tree or hash index
WHERE department_id = ? AND salary > ?	Composite B+ tree on (department_id, salary)
Join on foreign key	Index on foreign key column
LIKE 'abc%'	B+ tree may help
LIKE '%abc'	Normal B+ tree usually cannot help

36. Index Selectivity

Selectivity measures how many rows are filtered by a condition.

High selectivity:

```
email = 'a@example.com'
```

This returns very few rows.

Low selectivity:

```
gender = 'M'
```

This may return half the table.

Formula:

```
selectivity = number of distinct values / total number of rows
```

Higher selectivity usually means a more useful index.

37. Cardinality

Cardinality means the number of distinct values in a column.

High-cardinality columns:

- `email`
- `phone_number`
- `roll_no`
- `employee_id`

Low-cardinality columns:

- `gender`
- `status`
- `boolean flags`

Indexes are usually more useful on high-cardinality columns.

38. Index Scan vs Table Scan

Table Scan

The DBMS reads the entire table.

Useful when:

- Table is small.
- Query returns most rows.
- No useful index exists.

Index Scan

The DBMS reads the index to find matching rows.

Useful when:

- Query returns a small number of rows.
- Index is selective.
- Index order helps sorting or grouping.

Index Seek

An index seek is more precise than an index scan. The DBMS directly navigates to a specific key or key range.

Example:

```
WHERE id = 10
```

with an index on `id`.

39. Page Splitting

In B-tree or B+ tree indexes, a page split happens when a page is full and a new key must be inserted.

The DBMS splits the page into two pages and updates parent nodes.

Why Page Splits Matter

- They cost extra I/O.
- They can cause fragmentation.
- They may reduce insertion performance.

Random inserts, such as random UUID primary keys, can cause frequent page splits.

40. Fragmentation

Index fragmentation occurs when index pages become physically scattered or partially filled due to many insertions, deletions, and updates.

Effects:

- More disk I/O.
- Slower range scans.
- Wasted storage.

Databases may provide maintenance operations such as:

- Rebuild index
- Reorganize index
- Vacuum/analyze, depending on DBMS

41. Indexing and Joins

Indexes can significantly improve join performance.

Example:

```
SELECT *  
FROM Student s  
JOIN Department d  
  ON s.department_id = d.department_id;
```

Useful indexes:

- `Department(department_id)` because it is likely a primary key.
- `Student(department_id)` because it is a foreign key.

Foreign key columns are often good index candidates because joins frequently use them.

42. Indexing and Sorting

An ordered index can help avoid sorting.

Example:

```
SELECT *  
FROM Employee  
ORDER BY salary;
```

An index on **salary** may return rows in sorted order.

Composite index:

```
CREATE INDEX idx_emp_dept_salary  
ON Employee(department_id, salary);
```

This can help:

```
SELECT *  
FROM Employee  
WHERE department_id = 10  
ORDER BY salary;
```

because within each **department_id**, rows are sorted by **salary**.

43. Indexing and Grouping

Indexes can help grouping if the data can be read in grouped order.

Example:

```
SELECT department_id, COUNT(*)  
FROM Employee  
GROUP BY department_id;
```

An index on **department_id** may help because equal department values are stored together in the index.

44. Indexing and LIKE

B+ tree index on **name** can help:

```
WHERE name LIKE 'Rah%'
```

because this is a prefix search.

It usually cannot help:

```
WHERE name LIKE '%Rah'
```

because the beginning of the string is unknown.

45. Indexing and NULL Values

DBMS behavior differs, but indexes may store NULL values.

Important points:

- Some DBMSs include NULL values in B-tree indexes.
- Unique indexes may allow multiple NULLs depending on DBMS.
- Queries using `IS NULL` may use an index if the DBMS stores NULLs in the index.

Example:

```
SELECT * FROM Employee WHERE manager_id IS NULL;
```

An index on `manager_id` may help if many rows are not NULL and only few are NULL.

46. Indexing and Updates

When a row is inserted:

- The table receives the row.
- Every relevant index must receive a new entry.

When a row is deleted:

- The table removes or marks the row.
- Every relevant index must remove or mark the entry.

When an indexed column is updated:

- The old index entry must be removed.
- The new index entry must be inserted.

Therefore, indexes speed up reads but slow down writes.

47. Choosing Columns for Indexing

Good candidates:

- Primary keys
- Foreign keys
- Columns used frequently in `WHERE`
- Columns used frequently in joins

- Columns used frequently in **ORDER BY**
- High-cardinality columns
- Columns used in range queries

Bad candidates:

- Columns rarely used in queries
- Low-cardinality columns
- Frequently updated columns
- Very large text columns
- Columns in small tables

48. Common Mistakes

Creating Too Many Indexes

Every index adds write overhead. More indexes are not always better.

Indexing Every Column

This wastes storage and slows writes.

Wrong Composite Index Order

Index (**salary**, **department_id**) is not the same as (**department_id**, **salary**).

Ignoring Query Patterns

Indexes should be based on actual queries, not just table structure.

Using Functions on Indexed Columns

```
WHERE YEAR(order_date) = 2026
```

This may prevent index usage.

Better:

```
WHERE order_date >= '2026-01-01'  
AND order_date < '2027-01-01'
```

49. Real DBMS Notes

MySQL InnoDB

- Uses clustered index for the primary key.
- If no primary key exists, it chooses a unique non-null key.
- If no suitable key exists, it creates a hidden clustered key.

- Secondary indexes store primary key values at leaf nodes.
- B+ tree indexes are commonly used.

PostgreSQL

- Tables are heap-organized by default.
- Primary key creates a unique B-tree index.
- Supports B-tree, hash, GiST, SP-GiST, GIN, and BRIN indexes.
- B-tree is the default index type.

SQL Server

- Supports clustered and non-clustered indexes.
- A table can have one clustered index.
- A table can have many non-clustered indexes.
- Primary key can be clustered or non-clustered.

Oracle

- Uses B-tree indexes commonly.
- Also supports bitmap indexes and function-based indexes.
- Index-organized tables store rows in a B-tree structure.

50. Important Placement Comparisons

Primary Index vs Clustered Index

They are related but not always identical.

In textbook DBMS:

- Primary index is built on the primary key or ordering key.
- It is often sparse.

In real DBMS:

- Clustered index determines physical/logical row storage order.
- A primary key may or may not be clustered depending on DBMS.

Secondary Index vs Non-Clustered Index

They are also related but not exactly the same.

- Secondary index means index on a non-ordering attribute.
- Non-clustered index means index separate from table data.

Many secondary indexes are non-clustered.

51. Quick Interview Answers

What is an index?

An index is an auxiliary data structure that stores search key values with pointers to actual records or blocks, allowing faster data retrieval.

Why are indexes used?

Indexes reduce disk I/O and speed up searches, joins, sorting, grouping, and constraint checking.

What is the main disadvantage of indexing?

Indexes require extra storage and slow down insert, delete, and update operations because index structures must also be maintained.

What is a primary index?

A primary index is built on a primary key or ordering key of a sorted data file. It is often sparse because one entry per block is enough.

What is a secondary index?

A secondary index is built on a non-ordering attribute. It provides an alternate access path and is usually dense.

What is a dense index?

A dense index has an index entry for every record or every search key value.

What is a sparse index?

A sparse index has entries only for some keys, usually one entry per data block. It requires the data file to be sorted on the search key.

What is a clustered index?

A clustered index determines the physical or logical order of rows in a table. A table usually has only one clustered index.

What is a non-clustered index?

A non-clustered index is stored separately from the table and contains keys with pointers to actual rows.

Why is B+ tree preferred over B-tree in DBMS?

B+ tree is preferred because all data pointers are stored at leaf nodes, internal nodes have high fanout, tree height is small, and linked leaves make range queries very efficient.

When is a hash index better than a B+ tree?

A hash index can be better for exact equality search, such as `id = 10`. It is not good for range queries or sorting.

What is a composite index?

A composite index is built on multiple columns. The order of columns matters, and B+ tree composite indexes follow the leftmost prefix rule.

52. Common Exam-Style MCQ Points

- Sparse index requires sorted data.
- Dense index can be used even if data is not sorted.
- Primary index is usually sparse.
- Secondary index is usually dense.
- Clustered index affects data storage order.
- Only one clustered index is generally possible per table.
- Many non-clustered indexes can exist.
- B+ tree is better than B-tree for range queries.
- Hash index is best for equality search.
- Hash index is poor for range search.
- Composite index column order matters.
- Indexes speed reads but slow writes.
- Indexes require extra storage.

53. One-Page Revision Table

Index Type	Main Idea	Best For	Weakness
Primary index	Index on primary/ordering key	Primary key lookup	Usually only one ordering path
Secondary index	Index on non-ordering column	Alternate search paths	More storage, usually dense
Dense index	Entry for every key/record	Fast lookup	Large size
Sparse index	Entry for some keys/blocks	Sorted files, less storage	Requires sorted data
Clustered index	Data stored in index order	Range queries, sorting	Only one, costly page splits
Non-clustered index	Separate index with row pointers	Multiple query patterns	Extra lookup may be needed
B-tree	Balanced multi-way tree	Ordered search	Range scan less efficient than B+ tree
B+ tree	Data pointers at linked leaves	Range queries, DBMS indexes	More complex
Hash index	Hash key to bucket	Equality lookup	Bad for range/sort
Composite index	Index on multiple columns	Multi-column filters	Column order matters

54. Memory Tricks

- Dense means "every key is present".

- Sparse means "some keys are present".
- Clustered means "data is stored together in index order".
- Non-clustered means "index is separate from table data".
- B+ tree means "plus linked leaves and all data at leaves".
- Hash index means "fast equality, bad range".
- Composite index means "combined columns, leftmost prefix matters".

55. Final Placement Summary

Indexing is one of the most important DBMS performance concepts. An index stores search keys and pointers so that records can be found without scanning the entire table. Primary indexes are usually built on ordering keys and are often sparse. Secondary indexes provide alternate access paths and are usually dense. Dense indexes store entries for every key or record, while sparse indexes store entries only for selected keys and require sorted data.

Clustered indexes organize the table according to the index key, making range queries efficient, while non-clustered indexes are separate structures that point to table rows. B-trees and B+ trees are balanced tree indexes, but B+ trees are preferred in DBMSs because their linked leaves support efficient range scans and sequential access. Hash indexes are excellent for equality searches but poor for range queries. Composite indexes use multiple columns, and their column order is critical because of the leftmost prefix rule.

The key tradeoff is simple: indexes make reads faster but writes slower and consume extra storage. Good indexing depends on query patterns, selectivity, cardinality, and workload.